# Day 33: Feature Selection and Dimensionality Reduction

Complete guide to filter, wrapper, and embedded methods, PCA, scikit-learn, exercises, and solutions

**Goal:** learn when to keep original columns, when to create fewer transformed components, and how to evaluate both approaches without test-data leakage.




**Dataset note:** the practice script uses scikit-learn's built-in Breast Cancer Wisconsin dataset for educational classification only. It is not a medical diagnostic tool.

## 1. What is a feature?

A feature is an input variable or column used by a model to make predictions. In a student example, study hours, attendance, and previous marks could be features. Pass/fail status is the target. A feature should be available at prediction time and should not directly reveal the target or future information.

## 2. Feature selection

Feature selection keeps a subset of original columns. It can reduce complexity, noise, and training cost, and may improve interpretability. But fewer features do not guarantee better performance. Always compare against a baseline using a suitable validation strategy.

## 3. Filter methods

Filter methods score features using statistics rather than repeatedly fitting the final predictive model. Examples include variance threshold, correlation, ANOVA F-score, chi-square tests, and mutual information. They are often fast but may miss interactions between features. Chi-square feature selection, for example, expects non-negative inputs.

```
from sklearn.feature_selection import SelectKBest, f_classif
selector = SelectKBest(score_func=f_classif, k=10)
selector.fit(X_train, y_train)
X_train_selected = selector.transform(X_train)
```

## 4. Wrapper methods

Wrapper methods evaluate candidate feature subsets using a model. Recursive Feature Elimination (RFE) repeatedly fits an estimator, removes less important features, and continues until the desired number remains. Sequential feature selection is another example. These methods can be effective but expensive on large datasets.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
selector = RFE(LogisticRegression(max_iter=5000), n_features_to_select=10)
selector.fit(X_train, y_train)
names = X_train.columns[selector.get_support()]
```

## 5. Embedded methods

Embedded methods select or shrink features during model training. L1-regularized linear models can shrink coefficients to zero. Tree-based models provide feature-importance scores that can be used with SelectFromModel. Importance describes how a model uses information, not whether a feature causes an outcome.

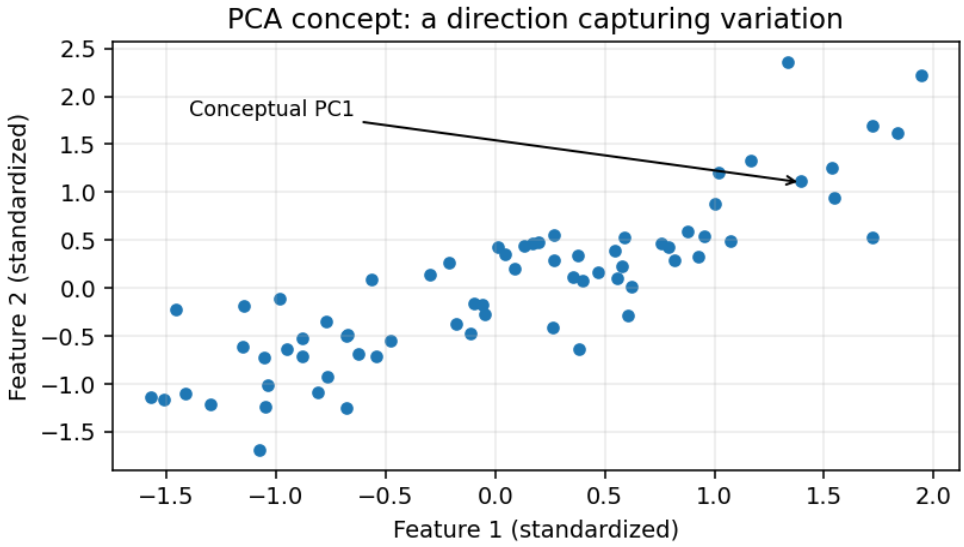
```
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier
selector = SelectFromModel(
    RandomForestClassifier(n_estimators=100, random_state=42),
    threshold="median"
)
selector.fit(X_train, y_train)
```

## 6. Compare selection methods

**Filter:** fast statistical ranking, may miss interactions. **Wrapper:** tests subsets using a model, often computationally expensive. **Embedded:** selection is integrated into estimator fitting and depends on the model's assumptions. No single method is best for every dataset.

## 7. Dimensionality reduction and PCA

Dimensionality reduction represents data with fewer dimensions. Feature selection retains original columns, while PCA creates new components from combinations of the original numerical features.




PCA finds orthogonal directions called principal components. PC1 captures the greatest possible variance. PC2 captures the greatest remaining variance subject to being perpendicular to PC1. Later components capture progressively less variance. Each component is a weighted combination of original features.

PCA is useful for compact representations, visualization, and some high-dimensional workflows. It is unsupervised and does not use the target to find components. Therefore, it can discard a low-variance direction that is important for prediction. Components can also be harder to interpret.

**Explained variance ratio:** the proportion of total variance represented by a component. PCA(n_components=0.95) retains enough components to represent at least 95% of the variance in the data used to fit PCA, within dimensional limits.

## 8. Why scale before PCA?

Features with large numeric ranges can dominate variance. StandardScaler learns the mean and standard deviation from training data and standardizes each column. Scaling is commonly useful before PCA when features have different units or ranges. It is not automatically required in every context.

## 9. Leakage-safe workflow

Split training and test data first. Fit scaling, feature selection, and PCA using training data only. During cross-validation, keep preprocessing inside a Pipeline so each fold learns its own transformations from its training portion. Use transform, not fit_transform, on the held-out test data.

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("select", SelectKBest(score_func=f_classif, k=10)),
    ("model", LogisticRegression(max_iter=5000))
])
pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```

A pipeline helps prevent preprocessing leakage, but it cannot automatically identify every form of target leakage, duplicate records, or time leakage.

## 10. Practice script overview

The attached script loads a built-in dataset, makes a stratified train/test split, trains a SelectKBest pipeline, prints selected feature names, runs five-fold cross-validation on training data, compares a PCA pipeline, and demonstrates RFE. Install pandas, scikit-learn, and compatible NumPy to run it.

**Key functions:** load_breast_cancer loads example data. train_test_split separates training and test data. f_classif scores features with an ANOVA F-test. get_support returns selected-column indicators. cross_val_score evaluates training folds.

PCA(n_components=0.95) chooses components by explained variance.

## 11. Common mistakes

• Selecting features on the full dataset before splitting. • Fitting PCA before the split. • Assuming correlation proves causation. • Assuming high variance means predictive usefulness. • Comparing methods on different test sets. • Relying only on accuracy when classes are imbalanced. • Treating an educational dataset as a diagnostic tool.

## 12. Exercises with solutions

**Q1. Feature selection vs PCA?**
Solution: feature selection keeps some original columns. PCA creates new components from combinations of original numerical features.

**Q2. Which approach evaluates feature subsets with a predictive model?**
Solution: wrapper methods, such as RFE.

**Q3. Name an embedded method.**
Solution: L1 regularization or selection using a fitted tree estimator.

**Q4. Why put selection in a Pipeline?**
Solution: it is fitted only on appropriate training data inside each cross-validation fold.

**Q5. What does PCA(n_components=0.95) request?**
Solution: enough components to retain at least 95% of variance in the data used to fit PCA.

**Q6. Does PCA guarantee better predictions?**
Solution: no. It preserves variance, not necessarily target-relevant information.

**Q7. Why is importance not causation?**
Solution: predictive importance can reflect correlations, confounding, or dataset structure.

## 13. Revision checklist

■ Define feature and target. ■ Explain filter, wrapper, and embedded methods. ■ Explain SelectKBest and RFE. ■ Distinguish original selected columns from PCA components. ■ Explain principal components and explained variance. ■ Fit selection and PCA only on training data. ■ Use Pipeline for cross-validation. ■ Compare with a baseline.

## 14. Final takeaway

The goal is not to minimize feature count at any cost. It is to find a simpler or more useful representation without losing important predictive information. Choose methods based on the data, avoid leakage, and evaluate changes carefully.

Suggested reference: scikit-learn User Guide topics on feature selection, PCA/decomposition, Pipeline, and model selection.